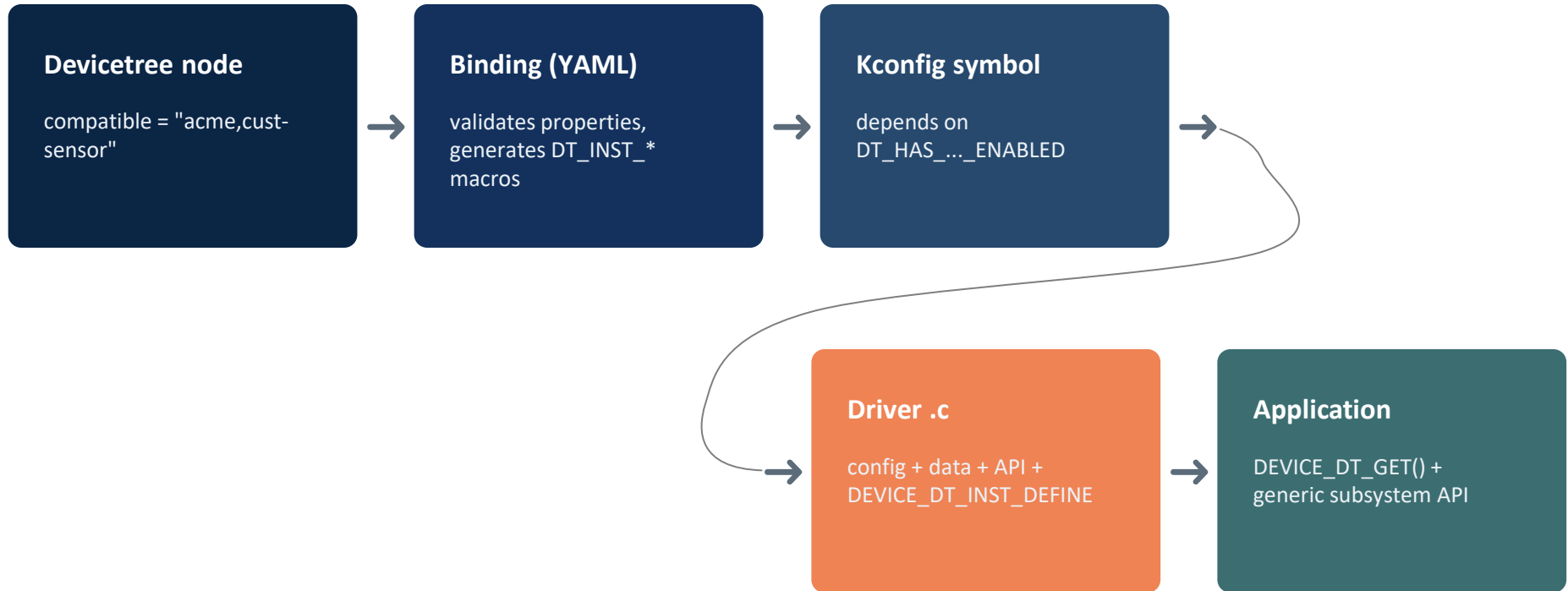


Creating a Custom Device Driver

The Zephyr Driver Model – end to end



Devicetree & bindings

- Devicetree describes hardware topology: buses, pins, addresses — never software behavior
- Every node is matched to a driver purely by its compatible string
- A binding (YAML) declares legal / required / optional properties
- **Options when writing a binding:**
 - `required: true` — fail fast if an overlay omits it
 - `default:` — make a property optional
 - `include: base.yaml` — inherit common properties (label, status)

```
compatible: "acme,cust-sensor"
include: base.yaml
properties:
  sim-min-mc:
    type: int
    required: true
  sim-max-mc:
    type: int
    required: true
  sim-step-mc:
    type: int
    default: 100
```

dts/bindings/sensor/acme,cust-sensor.yaml

Kconfig: build-time configuration

- Controls whether the driver is compiled in, and build-time trade-offs
- Modern pattern: gate on the auto-generated `DT_HAS_<COMPAT>_ENABLED` symbol
- The option only appears once a matching devicetree node exists
- No manual bookkeeping — devicetree and Kconfig stay in sync automatically

```
config CUST_SENSOR
    bool "ACME custom simulated
        temperature sensor"
    default y
    depends on
        DT_HAS_ACME_CUST_SENSOR_ENABLED
    select SENSOR
```

drivers/sensor/cust_sensor/Kconfig

Anatomy of a driver instance

Config struct

Read-only, filled from devicetree at build time (pins, addresses, calibration). Lives in ROM/flash.

Data struct

Mutable runtime state (last reading, counters, mutexes). One instance per device, in RAM.

API struct

A function-pointer table implementing a subsystem contract (e.g. `sensor_driver_api`). Enables generic calls from application code.

Init function

Runs once at boot; should probe hardware and return a negative `errno` on failure.

Where can a custom driver live?

Option	Driver location	Wiring mechanism	Best fit
1. App-local / freestanding folder	Inside or next to the app	ZEPHYR_EXTRA_MODULES (relative path)	Prototyping, single-app drivers
2. Freestanding folder elsewhere	Shared drive / sibling checkout	Same, absolute path	Sharing across a few local apps
3. West workspace module	Its own git repository	project: entry in west.yml manifest	Reuse, versioning, CI reproducibility
4. In-tree	zephyr/drivers/<class>/	Native CMake/Kconfig chain	Only for upstream contribution

The driver's source code doesn't change between options 1–3 — only how it's discovered by the build system.

zephyr/module.yml — the file that makes it portable

```
# <driver-root>/zephyr/module.yml
build:
  cmake: .
  kconfig: Kconfig
settings:
  dts_root: .
```

- build.cmake — where to find the module's CMakeLists.txt
- build.kconfig — where to find its Kconfig entry point
- settings.dts_root — adds the module as a devicetree binding search root
- **This single file is identical whether the folder sits next to your app, on a shared drive, or is the root of its own git repo**

Option 1 & 2 — ZEPHYR_EXTRA_MODULES

```
# app/CMakeLists.txt
# – before find_package(Zephyr ...)
set(ZEPHYR_EXTRA_MODULES
    ${CMAKE_CURRENT_SOURCE_DIR}/../cust_sensor_module)

find_package(Zephyr REQUIRED
    HINTS $ENV{ZEPHYR_BASE})
```

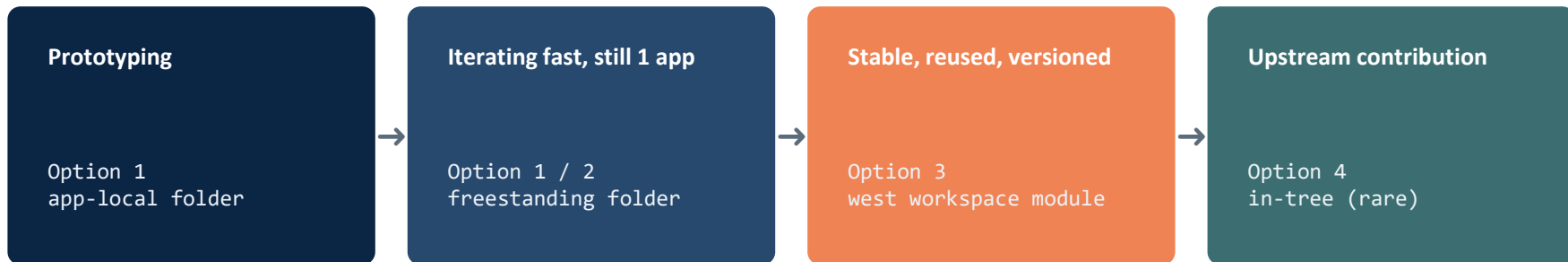
- **Set before `find_package(Zephyr ...)` — order matters**
- Treated exactly as if west had discovered the module
- Option 1: relative path — driver folder ships inside the app repo
- Option 2: absolute path — driver folder lives elsewhere on disk
- Multiple modules: separate with ";" or spaces

Option 3 — West workspace / manifest module

```
# west.yml (excerpt)
manifest:
  projects:
    - name: cust-sensor-driver
      url: https://git.example.com/
        acme/cust_sensor_module
      revision: v1.0.0
      path: modules/cust_sensor_driver
```

- Driver published as its own git repository
- west update fetches it automatically
- module.yml is auto-discovered — no ZEPHYR_EXTRA_MODULES needed
- revision: pins an exact, reproducible version
- **Best for: reuse across teams/products, code review, CI**

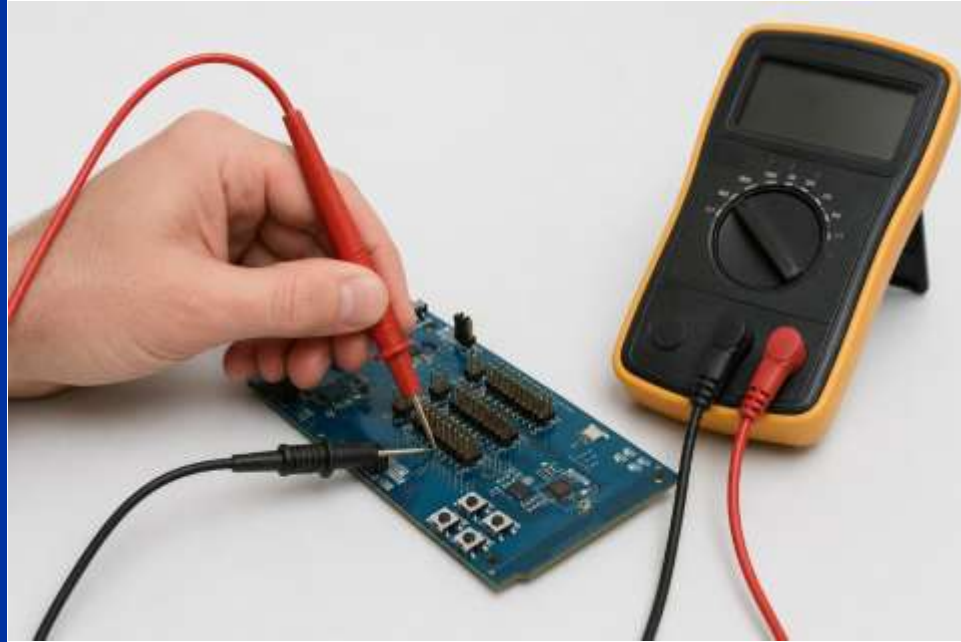
Which option should you start with?



Migration between options only ever changes one line of CMake/manifest — never the driver source.

Hands-on

- Creating a Custom Device Driver



Step 1 – Creating a custom driver

See Dev Academy Course *nRF Connect SDK Intermediate*

=> *Lesson 7: Device Driver Development*